

Отчёт по лабораторной работе №13

Операционные системы

Гасанова Шакира Чингизовна

Содержание

Цель	5
Задание	6
Выполнение лабораторной работы	8
Контрольные вопросы	13
Выводы	17
Список литературы	18

Список иллюстраций

1	Первый скрипт	8
2	Запуск файла	9
3	Проверка	9
4	Программа	9
5	Второй скрипт	10
6	Проверка	10
7	Третий скрипт	11
8	Проверка	11
9	Четвёртый скрипт	11
10	Исполнение файла	12
11	Архив	12
12	Модификация скрипта	12
13	Исполнение файла	12

Список таблиц

Цель

Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

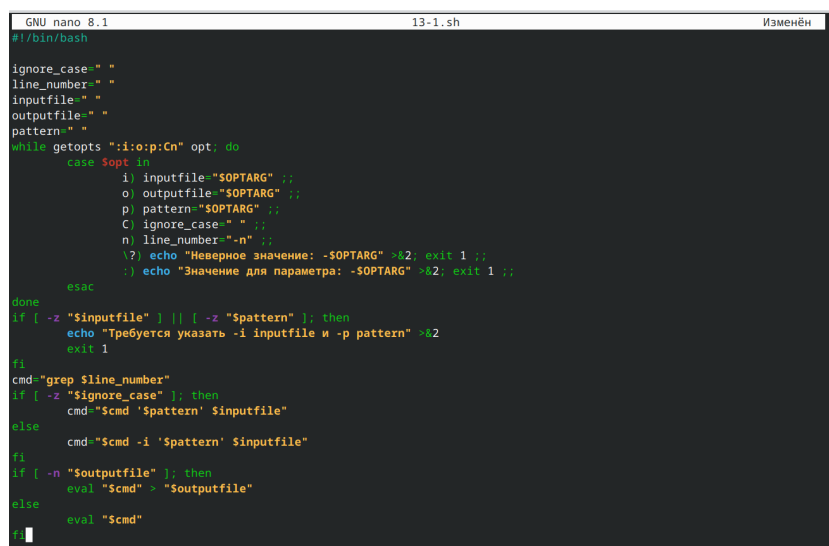
Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами:
 - `iinputfile`
 - `ooutputfile`
 - `р`шаблон
 - `С`
 - `п` а затем ищет в указанном файле нужные строки, определяемые ключом `-р`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в `о` коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N (например `1.tmp`, `2.tmp`, `3.tmp`, `4.tmp` и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так,

чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

Выполнение лабораторной работы

Создаю файл 13-1.sh и пишу там скрипт, который анализирует командную строку с приведёнными ключами, а затем ищет в указанном файле нужные строки (рис. @fig:001).



```
GNU nano 8.1 13-1.sh
#!/bin/bash

ignore_case=" "
line_number=" "
inputfile=" "
outputfile=" "
pattern=" "

while getopts ":i:o:p:C:n" opt; do
  case $opt in
    i) inputfile="$OPTARG" ;;
    o) outputfile="$OPTARG" ;;
    p) pattern="$OPTARG" ;;
    C) ignore_case=" " ;;
    n) line_number="-n" ;;
    \?) echo "Неверное значение: -$OPTARG" >&2; exit 1 ;;
    :) echo "Значение для параметра: -$OPTARG" >&2; exit 1 ;;
  esac
done

if [ -z "$inputfile" ] || [ -z "$pattern" ]; then
  echo "Требуется указать -i inputfile и -p pattern" >&2
  exit 1
fi

cmd="grep $line_number"
if [ -z "$ignore_case" ]; then
  cmd="$cmd '$pattern' $inputfile"
else
  cmd="$cmd -i '$pattern' $inputfile"
fi

if [ -n "$outputfile" ]; then
  eval "$cmd" > "$outputfile"
else
  eval "$cmd"
fi
```

Рис. 1: Первый скрипт

Делаю файл исполняемым, проверяю работоспособность (рис. @fig:002, рис. @fig:003).


```
shakiragas@fedora:~$ nano 13-1.sh
shakiragas@fedora:~$ chmod +x 13-1.sh
shakiragas@fedora:~$ touch inputfile.txt
shakiragas@fedora:~$ touch outputfile.txt
shakiragas@fedora:~$ ./13-1.sh -i inputfile.txt -p "крыльцо" -o outputfile.txt -n
bash: ./13-1.sh: Нет такого файла или каталога
shakiragas@fedora:~$ ./13-1.sh -i inputfile.txt -p "крыльцо" -o outputfile.txt -n
grep: inputfile.txt: Нет такого файла или каталога
shakiragas@fedora:~$ ./13-1.sh -i inputfile.txt -p "крыльцо" -o outputfile.txt -n
shakiragas@fedora:~$
```

Рис. 2: Запуск файла

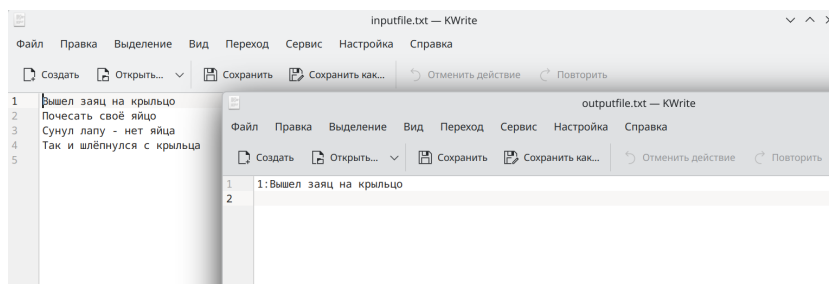


Рис. 3: Проверка

Создаю файл 13-2.c и пишу на языке Си программу, которая выводит число и сравнивает его с нулём (рис. @fig:005).

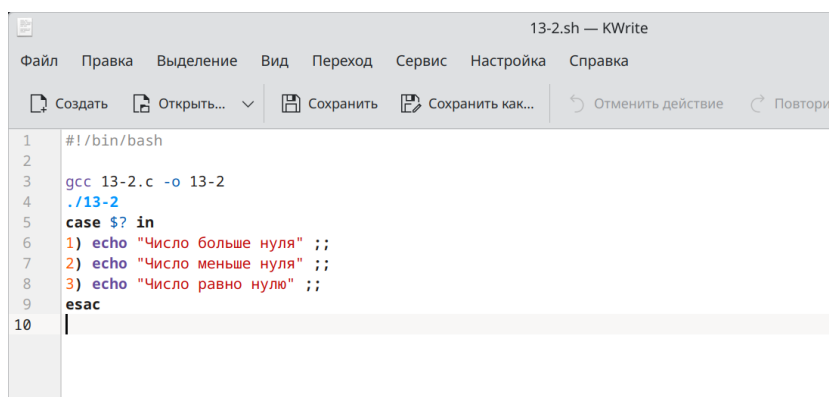


Рис. 4: Программа

Пишу командный файл 13-2.sh, который вызывает программу (13-2.c) и выдаёт сообщение о том, какое число было введено (рис. @fig:004).

```

GNU nano 8.1                                     13-2.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;
    printf("Введите число: ");
    scanf("%d", &num);
    if (num > 0)
        exit(1);
    else if (num < 0)
        exit(2);
    else
        exit(3);
}

```

Рис. 5: Второй скрипт

Делаю файл исполняемым, проверяю работу (рис. @fig:006).

```

shakiragas@fedora:~$ nano 13-2.c
shakiragas@fedora:~$ nano 13-2.sh
shakiragas@fedora:~$ chmod +x 13-2.sh
shakiragas@fedora:~$ ./13-2.sh
./13-2.sh: строка 3: gss: команда не найдена
./13-2.sh: строка 4: ./cprog: Нет такого файла или каталога
shakiragas@fedora:~$ ./13-2.sh
Введите число: 3
Число больше нуля
shakiragas@fedora:~$ ./13-2.sh
Введите число: 0
Число равно нулю
shakiragas@fedora:~$ ./13-2.sh
Введите число: -12
Число меньше нуля
shakiragas@fedora:~$

```

Рис. 6: Проверка

Создаю файл 13-3.sh и пишу командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N, а затем удаляет их (рис. @fig:007).

```

GNU nano 8.1                                     13-3.sh
#!/bin/bash

if [ "$1" == "create" ]; then
    N=$2
    if [ -z "$N" ]; then
        echo "Введите количество файлов "
        exit 1
    fi
    for ((i=1; i<=N; i++))
    do
        touch "${i}.tmp"
    done
elif [ "$1" == "delete" ]; then
    rm -f *.tmp
else
    echo "Выберите, что сделать"
fi

```

Рис. 7: Третий скрипт

Делаю файл исполняемым, проверяю (рис. @fig:008).

```

shakiragas@fedora: $ chmod +x 13-3.sh
shakiragas@fedora: $ ./13-3.sh create 3
shakiragas@fedora: $ ls
12-1.sh  13-2.c      2.tmp    git-extended  output.txt  Загрузки
12-2.sh  13-2.sh     3.tmp    inputfile.txt shakiragas.github.io  Изображения
12-3.sh  13-3.sh     backup   lab11.sh     text.txt   Музыка
12-4.sh  1.tmp       bin      LICENSE      work       Общедоступные
13-1.sh  '2025-05-01 19-27-48.mkv' conf.txt  logfiles     outputfile.txt  Документы
13-2    '2025-05-09 18-15-39.mkv' file.txt  outputfile.txt
shakiragas@fedora: $ ./13-3.sh delete
shakiragas@fedora: $ ls
12-1.sh  13-2.c      bin      LICENSE      work       Общедоступные
12-2.sh  13-2.sh     conf.txt logfiles     outputfile.txt  Документы
12-3.sh  13-3.sh     file.txt outputfile.txt  shakiragas.github.io  Изображения
12-4.sh  '2025-05-01 19-27-48.mkv' git-extended  output.txt  Загрузки
13-1.sh  '2025-05-09 18-15-39.mkv' inputfile.txt shakiragas.github.io  Изображения
13-2    backup     lab11.sh  text.txt     Музыка
shakiragas@fedora: $

```

Рис. 8: Проверка

Пишу командный файл 13-4.sh, который с помощью команды tar запаковывает в архив все файлы в указанной директории (рис. @fig:009).

```

GNU nano 8.1                                     13-4.sh
#!/bin/bash

directory="$1"
archive="$2"
if [ -z "$directory" ] || [ -z "$archive" ]; then
    exit 1
fi
tar -czf "$archive" -C "$directory" .

```

Рис. 9: Четвёртый скрипт

Проверяю созданный архив (рис. @fig:010, рис. @fig:011).

```
shakiragas@fedora:~$ chmod +x 13-4.sh
shakiragas@fedora:~$ ./13-4.sh 13 backup1.tar.gz
shakiragas@fedora:~$
```

Рис. 10: Исполнение файла

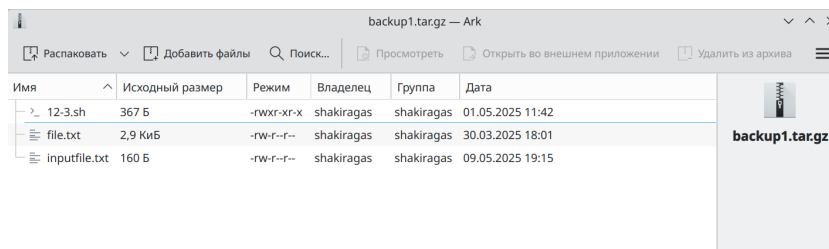


Рис. 11: Архив

Модифицирую файл, чтобы запаковывал только те файлы, которые были изменены за последнюю неделю (рис. @fig:012).

```
GNU nano 8.1 13-4.sh
#!/bin/bash

directory="$1"
archive="$2"
if [ -z "$directory" ] || [ -z "$archive" ]; then
    exit 1
fi
find "$directory" -type f -mtime -7 | tar -czf "$archive" -T -
```

Рис. 12: Модификация скрипта

Проверяю созданный архив (рис. @fig:013).

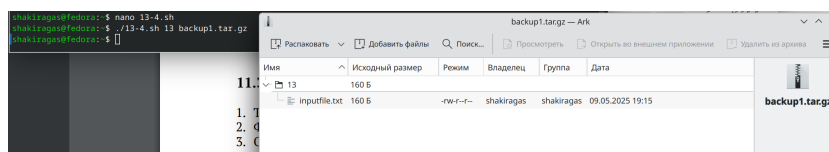


Рис. 13: Исполнение файла

Контрольные вопросы

1. Каково предназначение команды `getopts`? Осуществляет синтаксический анализ командной строки, выделяя флаги, и используется для объявления переменных. Синтаксис команды следующий: `getopts option-string variable`. Флаги – это опции командной строки, обычно помеченные знаком минус; Например, `-F` является флагом для команды `ls -F`. Иногда эти флаги имеют аргументы, связанные с ними. Программы интерпретируют эти флаги, соответствующим образом изменяя свое поведение. Строка опций `option-string` — это список возможных букв и чисел соответствующего флага. Если ожидается, что некоторый флаг будет сопровождаться некоторым аргументом, то за этой буквой должно следовать двоеточие. Соответствующей переменной присваивается буква данной опции. Если команда `getopts` может распознать аргумент, она возвращает истину. Принято включать `getopts` в цикл `while` и анализировать введенные данные с помощью оператора `case`. Предположим, необходимо распознать командную строку следующего формата: `testprog -ifile_in.txt -ofile_out.doc -L -t -r` Вот как выглядит использование оператора `getopts` в этом случае:

```
while getopts o: i:Ltr optletter do case $optletter in o) oflag = 1; oval =OPTARG;; i) iflag=1; ival=$OPTARG;; L) Lflag=1;; t) tflag=1;; r) rflag=1;; *) echo Illegal option $optletter esac done
```

 Функция `getopts` включает две специальные переменные среды – `OPTARG` и `OPTIND`. Если ожидается дополнительное значение, то `OPTARG` устанавливается в значение этого аргумента (будет равна `file_in.txt` для опции `i` и `file_out.doc` для опции `o`). `OPTIND` является

числовым индексом на упомянутый аргумент. Функция `getopts` также понимает переменные типа массив, следовательно, можно использовать ее в функции не только для синтаксического анализа аргументов функций, но и для анализа введенных пользователем данных.

2. Какое отношение метасимволы имеют к генерации имён файлов? При перечислении имён файлов текущего каталога можно использовать следующие символы: `*` – соответствует произвольной, в том числе и пустой строке; `?` – соответствует любому одинарному символу; `[c1-c2]` – соответствует любому символу, лексикографически находящемуся между символами `c1` и `c2`. Например, `echo *` – выведет имена всех файлов текущего каталога, что представляет собой простейший аналог команды `ls`; `ls .c` – выведет все файлы с последними двумя символами, совпадающими с `.c`. `echo prog.?` – выведет все файлы, состоящие из пяти или шести символов, первыми пятью символами которых являются `prog.` `[a-z]` – соответствует произвольному имени файла в текущем каталоге, начинающемуся с любой строчной буквы латинского алфавита.
3. Какие операторы управления действиями вы знаете? Часто бывает необходимо обеспечить проведение каких-либо действий циклически и управление дальнейшими действиями в зависимости отрезультатов проверки некоторого условия. Для решения подобных задач язык программирования `bash` предоставляет возможность использовать такие управляющие конструкции, как `for`, `case`, `if` и `while`. С точки зрения командного процессора эти управляющие конструкции являются обычными командами и могут использоваться как при создании командных файлов, так и при работе в интерактивном режиме. Команды, реализующие подобные конструкции, по сути, являются операторами языка программирования `bash`. Поэтому при описании языка программирования `bash` термин оператор будет использоваться наравне с термином команда.

Команды ОС UNIX возвращают код завершения, значение которого может быть использовано для принятия решения о дальнейших действиях. Команда `test`, например, создана специально для использования в командных файлах. Единственная функция этой команды заключается в выработке кода завершения.

4. Какие операторы используются для прерывания цикла? Два несложных способа позволяют вам прерывать циклы в оболочке `bash`. Команда `break` завершает выполнение цикла, а команда `continue` завершает данную итерацию блока операторов. Команда `break` полезна для завершения цикла `while` в ситуациях, когда условие перестаёт быть правильным. Команда `continue` используется в ситуациях, когда больше нет необходимости выполнять блок операторов, но вы можете захотеть продолжить проверять данный блок на других условных выражениях.
5. Для чего нужны команды `false` и `true`? Следующие две команды ОС UNIX используются только совместно с управляющими конструкциями языка программирования `bash`: это команда `true`, которая всегда возвращает код завершения, равный нулю (т.е. истина), и команда `false`, которая всегда возвращает код завершения, не равный нулю (т. е. ложь).
6. Что означает строка `if test -f mans/i.s, ?if test -f mans/i.s` проверяет, существует ли файл `mans/i.s` и является ли этот файл обычным файлом. Если данный файл является каталогом, то команда вернет нулевое значение (ложь).
7. Объясните различия между конструкциями `while` и `until`. Выполнение оператора цикла `while` сводится к тому, что сначала выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, а затем, если последняя выполненная команда из этой последовательности команд возвращает

нулевой код завершения (истина), выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `do`, после чего осуществляется безусловный переход на начало оператора цикла `while`. Выход из цикла будет осуществлён тогда, когда последняя выполненная команда из последовательности команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово `while`, возвратит ненулевой код завершения (ложь). При замене в операторе цикла `while` служебного слова `while` на `until` условие, при выполнении которого осуществляется выход из цикла, меняется на противоположное. В остальном оператор цикла `while` и оператор цикла `until` идентичны.

Выводы

При выполнении данной лабораторной работы я научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Список литературы

1. Лабораторная работа №13 [Электронный ресурс]